

CharactUHC Plugin

Documentation des fichiers JSON

Gamemodes · Listes de rôles · Rôles · Actions · Jauges

Avant-propos — Comment lire ce document

Le charactUHC se configure entièrement via des fichiers texte au format JSON. Si tu n'es pas familier avec ce format, voici l'essentiel à retenir :

- Un fichier JSON est un fichier texte structuré avec des accolades { }, des crochets [] et des guillemets "".
- Chaque « propriété » est une paire clé : valeur, séparée des suivantes par une virgule.
- Les accolades { } délimitent un objet (un bloc de propriétés). Les crochets [] délimitent une liste d'éléments.

Tu n'as pas besoin de tout comprendre du JSON pour utiliser ce plugin. Suis simplement les exemples donnés dans ce document et adapte les valeurs.

Ce document utilise les conventions suivantes :

String	Une chaîne de caractères, toujours entre guillemets. Exemple : "Guerrier"
Integer	Un nombre entier (sans virgule). Exemple : 5
Number	Un nombre décimal (avec point). Exemple : 1.5
Boolean	Une valeur vraie ou fausse : true ou false (sans guillemets)
Object	Un bloc entre accolades { } contenant d'autres propriétés
Array	Une liste entre crochets [] contenant plusieurs éléments

Emplacements des fichiers : **Gamemodes/<nom>.json** pour les gamemodes, **roles/<nom>.json** pour les rôles.

1. Le fichier Gamemode

Le fichier gamemode configure la partie dans sa globalité : est-ce qu'il y a du minage, combien de temps, et quels groupes de rôles sont utilisés. Un seul fichier gamemode suffit par partie.

1.1 Exemple commenté

```
{
  // Active le système de minage (ressources bonus en début de partie)
  "hasMinageinage": true,

  // La phase de minage dure 20 minutes (ou 20s, à vérifier)
  "minageTime": 20,

  // Les ressources de minage sont doublées
  "boostMinage": 2,

  // Deux listes de rôles : chaque joueur recevra UN rôle de chaque liste
  "roleListNames": [
    { "listName": "classes", "timeApplication": 10 },
    { "listName": "origines", "timeApplication": 10 }
  ]
}
```

1.2 Toutes les propriétés

Clé	Type de valeur	Valeur par défaut	Description
hasMinageinage	Boolean	false	Active (true) ou désactive (false) le système de minage bonus en début de partie.
minageTime	Integer	10	Durée en minutes de la phase de minage. Passé ce délai, les ressources bonus s'arrêtent.
boostMinage	Integer	1	Multiplicateur de ressources pendant la phase de minage. 1 = normal, 2 = double, etc.
roleListNames	Array	[]	Liste des groupes de rôles actifs dans la partie. Voir section 2 pour le détail.
listName	String	"unreferenced"	(Dans roleListNames) Nom du groupe. Doit correspondre au roleListName des fichiers de rôle voulus.
timeApplication	Integer	aléatoire 3–20	(Dans roleListNames) Délai en minutes avant que les rôles de ce groupe soient révélés ou appliqués.
conditions	Object	aucune	Définit les conditions globales de la partie. C'est un objet dont chaque clé est le nom d'une condition et la valeur un booléen (true/false). Ces conditions peuvent être lues et modifiées par les Actions (voir type conditionChanged et filtre conditions).

2. Les listes de rôles

Une liste de rôles est une « couche » de rôles appliquée à tous les joueurs. Concrètement : si tu declares deux listes dans le gamemode, chaque joueur reçoit un rôle issu de la première liste ET un rôle issu de la deuxième liste, simultanément.

Exemple concret : une liste « classes » (Guerrier, Mage, Archer...) et une liste « origines » (Humain, Elfe, Nain...). Chaque joueur sera par exemple un « Mage Elfe » ou un « Guerrier Nain » — les deux rôles s'appliquent en même temps et leurs effets se cumulent.

```
// Dans le gamemode : deux listes déclarées
"roleListNames": [
  { "listName": "classes", "timeApplication": 10 },
  { "listName": "origines", "timeApplication": 10 }
]

// Résultat pour chaque joueur :
// → Joueur A : un rôle de "classes" + un rôle de "origines"
// → Joueur B : un rôle de "classes" + un rôle de "origines"
// → (les rôles attribués sont différents d'un joueur à l'autre)
```

💡 Pour qu'un rôle soit inclus dans une liste, il faut que sa propriété `roleListName` (dans son fichier de rôle) corresponde exactement au `listName` déclaré dans le gamemode.

⚠ Si le dossier `roles/` ne contient pas assez de fichiers avec le bon `roleListName`, certains joueurs pourraient ne pas recevoir de rôle pour cette liste. Prévois autant de rôles que de joueurs maximum par liste.

2.1 Paramètres d'une liste (définis dans le gamemode)

Clé	Type de valeur	Valeur par défaut	Description
<code>listName</code>	String	"unreferenced"	Identifiant du groupe. Les fichiers de rôle dont <code>roleListName</code> correspond à cette valeur seront distribués aux joueurs.
<code>timeApplication</code>	Integer	aléatoire 3–20	Délai en minutes avant l'application automatique des rôles de ce groupe. Passé ce délai, chaque joueur reçoit son rôle.

3. Le fichier Rôle

Chaque rôle est un fichier JSON dans le dossier roles/. Il définit tout ce que fait un personnage : ses statistiques de combat, ses conditions de résurrection, les actions qu'il déclenche (à l'attaque, à la mort, via des commandes), ses jauges et ses pouvoirs d'information.

! Un joueur peut cumuler plusieurs rôles (un par liste). Les effets de chaque rôle s'appliquent indépendamment.

3.1 Structure complète

```
{
  "name": "Mage", // Nom du rôle
  "camp": "Neutres", // Camp du joueur
  "roleListName": "classes", // Liste à laquelle appartient ce rôle

  "strenght": 1.2, // Modificateur de dégâts infligés
  "resistance": 0.8, // Modificateur de dégâts reçus
  "boostHealth": 4, // Points de vie bonus

  "reliveTry": 1, // Nombre de résurrections
  "lostStrenght": 0.1, // Malus de force à la résurrection
  "lostResis": 0.1, // Malus de résistance à la résurrection
  "lostHealth": 2.0, // PV perdus à la résurrection
  "conditionKilledBy": "", // Rôle du tueur requis pour ressusciter
  "conditionKilledByCamp": "", // Camp du tueur requis pour ressusciter

  "attackAction": { /* Action */ }, // Déclenchée à chaque attaque
  "deathAction": { /* Action */ }, // Déclenchée à la mort
  "commands": [ // Commandes personnalisées
    { "command": "freeze", "action": { /* Action */ } }
  ],
  "jauges": [ { /* Jauge */ } ], // Jauges (compteurs)
  "infoPowers": [ // Pouvoirs d'information
    { "type": "seeCaract", "object": { /* VisionPower */ } }
  ]
}
```

3.2 Identité et appartenance

Clé	Type de valeur	Valeur par défaut	Description
name	String	—	Nom du rôle tel qu'il sera affiché aux joueurs. Obligatoire.
camp	String	—	Nom du camp du joueur. Obligatoire. Utilisé pour les conditions de résurrection (conditionKilledByCamp) et le filtrage des actions (campTargeted).

Clé	Type de valeur	Valeur par défaut	Description
<code>roleListName</code>	String	"notReferenced"	Nom de la liste de rôles à laquelle ce rôle appartient. Doit correspondre au <code>listName</code> déclaré dans le gamemode pour que le rôle soit distribué.

3.3 Statistiques de combat

Ces valeurs modifient les dégâts infligés et reçus par le joueur. Une valeur de 1.0 signifie « normal, sans modification ». 1.5 signifie +50 %, 0.5 signifie -50 %.

Clé	Type de valeur	Valeur par défaut	Description
<code>strenght</code>	Number	1.0	Modificateur de dégâts infligés. Les dégâts finaux sont calculés ainsi : $\text{dégâts} \times (1 + \text{strenght})$. Exemple : <code>strenght = 0.5</code> → les dégâts sont multipliés par 1.5, soit +50 %.
<code>resistance</code>	Number	1.0	Modificateur de dégâts reçus. Formule identique : $\text{dégâts} \times (1 - \text{resistance})$. Exemple : <code>resistance = 0.3</code> → le joueur reçoit 70 % des dégâts normaux.
<code>boostHealth</code>	Integer	0	Points de vie supplémentaires accordés au joueur dès l'attribution du rôle. S'ajoute aux 20 PV de base de Minecraft.

3.4 Résurrection

Le système de résurrection permet à un joueur de revenir à la vie après sa mort, sous certaines conditions, et avec d'éventuels malus permanents.

Clé	Type de valeur	Valeur par défaut	Description
<code>reliveTry</code>	Integer	0	Nombre de fois que le joueur peut ressusciter. 0 = pas de résurrection possible.
<code>lostStrenght</code>	Number	0.0	Réduction de la valeur <code>strenght</code> à chaque résurrection. Exemple : si <code>strenght</code> vaut 1.2 et <code>lostStrenght</code> vaut 0.1, après une résurrection <code>strenght</code> sera de 1.1.
<code>lostResis</code>	Number	0.0	Réduction de la valeur <code>resistance</code> à chaque résurrection.
<code>lostHealth</code>	Number	0.0	Points de vie retirés définitivement à chaque résurrection.
<code>conditionKilledBy</code>	String	""	Nom du rôle que doit avoir le tueur pour que la résurrection soit possible. Laisser vide ("") pour n'imposer aucune condition.

Clé	Type de valeur	Valeur par défaut	Description
<code>conditionKilledByCamp</code>	String	""	Nom du camp que doit avoir le tueur pour que la résurrection soit possible. Laisser vide ("") pour n'imposer aucune condition.

Si les deux conditions sont renseignées, elles doivent être toutes les deux satisfaites. conditionKilledBy est vérifiée en premier.

3.5 Actions déclenchées automatiquement

Ces propriétés associent une Action (voir section 4) à des événements automatiques. Le format de chaque valeur est un objet Action complet.

Clé	Type de valeur	Valeur par défaut	Description
<code>attackAction</code>	Object (Action)	aucune	Action déclenchée automatiquement à chaque fois que le joueur inflige des dégâts. Le premier argument transmis est le nom de la victime, utilisable avec <code>commandTarget</code> pour la cibler directement.
<code>timedActions</code>	Array	[]	Liste d'actions planifiées à des moments précis. Chaque entrée est { "time": N, "action": { ... } } où time est le nombre de minutes depuis le début de la partie. Quand l'horloge atteint ce temps, l'action se déclenche pour chaque joueur portant ce rôle.
<code>deathAction</code>	Object (Action)	aucune	Action déclenchée automatiquement au moment où le joueur meurt.
<code>killAction</code>	Object (Action)	aucune	Action déclenchée automatiquement quand ce joueur tue un autre joueur. Contrairement à <code>deathAction</code> et <code>attackAction</code> , elle se déclenche du point de vue du tueur, pas de la victime.
<code>damageAction</code>	Object (Action)	aucune	Action déclenchée quand ce joueur reçoit des dégâts. Le premier argument transmis est le nom de l'attaquant, utilisable avec <code>commandTarget</code> pour le cibler directement.

3.6 Commandes personnalisées

Le tableau `commands` permet de créer des compétences activables manuellement via la commande `/lg`. Chaque entrée associe un mot-clé à une Action.

```
"commands": [
  {
    "command": "freeze",           // La commande sera : /lg freeze [arguments]
    "action": { /* Action */ }    // L'action déclenchée
  }
]
```

💡 Les arguments écrits après la commande (les noms de joueurs par exemple) sont transmis à l'Action. Utilise `commandTarget` ou `multipleCommandTarget` dans l'Action pour en tirer parti (voir section 4).

3.7 Jauges

Les jauges sont des compteurs associés à ce rôle. Elles permettent de déclencher une Action une fois qu'un certain seuil est atteint. Voir la section 5 pour le format détaillé.

```
"jauges": [
  { /* Objet Jauge — voir section 5 */ }
]
```

3.8 Pouvoirs d'information (infoPowers)

Les infoPowers permettent à un joueur d'obtenir des renseignements sur d'autres joueurs (leur camp, leur rôle, leurs statistiques...). Chaque pouvoir a un type et un objet de configuration.

Type disponible : seeCaract

Ce type permet de « lire » une propriété du fichier JSON d'un autre joueur. Par exemple, connaître son camp, son nombre de résurrections restantes, sa force, etc.

Clé	Type de valeur	Valeur par défaut	Description
<code>distance</code>	Integer	1000	Distance maximale en blocs à laquelle le pouvoir peut s'activer. Un joueur trop loin ne peut pas être ciblé.
<code>probaTrue</code>	Integer	100	Probabilité (en %) que l'information retournée soit la vraie valeur. Si le dé rate, une valeur aléatoire incorrecte est renvoyée à la place.
<code>use</code>	Integer	1	Nombre de fois que le pouvoir peut être utilisé au total.
<code>all</code>	Boolean	false	false = le pouvoir ne cible qu'un seul joueur à la fois. true = il s'applique à tous les joueurs dans le rayon en une seule activation.
<code>command</code>	String	""	Mot-clé pour activer le pouvoir manuellement via <code>/lg [command] [joueur]</code> . Si vide, le pouvoir ne peut pas être déclenché par commande.
<code>knowWho</code>	Boolean	false	false = le joueur qui utilise le pouvoir ne sait pas sur qui il s'est déclenché. true = il est informé du nom du joueur ciblé.
<code>caract</code>	String	"name"	Nom de la propriété JSON à observer dans le rôle de la cible. Exemples : "camp", "reliveTry", "strenght", "name".
<code>timeAlea</code>	Boolean	false	true = le pouvoir a 0,1 % de chance de se déclencher automatiquement chaque seconde (soit environ une

Clé	Type de valeur	Valeur par défaut	Description
			fois toutes les 17 minutes), sans que le joueur n'ait rien à faire.

```
// Exemple : pouvoir d'espionnage du camp via /lg espion <joueur>
{
  "type": "seeCaract",
  "object": {
    "distance": 50,
    "probaTrue": 75,    // 75 % de chance d'avoir la vraie info
    "use": 2,
    "all": false,
    "command": "espion",
    "knowWho": true,
    "caract": "camp",
    "timeAlea": false
  }
}
```


4. L'objet Action

Une Action est un bloc JSON qui décrit un effet appliqué à un ou plusieurs joueurs. Elle peut être utilisée dans `attackAction`, `deathAction`, dans une commande, dans une jauge, ou même imbriquée dans une autre Action via la clé `actions`.

```
{
  "type": "damage", // Type d'effet (voir liste complète)
  "force": 4.0, // Intensité
  "timeForce": 100, // Durée en ticks (pour les effets de
  potion)
  "proba": 80, // 80 % de chances que l'action se déclenche

  // --- Qui est ciblé ? ---
  "onlySelf": false, // Si true : seulement le lanceur
  "self": false, // Si true : inclure le lanceur dans les
  cibles
  "distance": 15, // Rayon en blocs
  "commandTarget": false, // Si true : cible = premier argument de la
  commande
  "multipleCommandTarget": false, // Si true : cibles = tous les arguments de
  la commande
  "campTargeted": "null", // Si renseigné : ne cibler que ce camp
  "roleTargeted": "null", // Si renseigné : ne cibler que ce rôle

  // --- Effets spéciaux ---
  "jauge": "null", // Nom de la jauge à incrémenter (type
  "jauge")
  "jaugeAdd": 0, // Valeur ajoutée à la jauge
  "actions": [] // Liste d'actions supplémentaires
  déclenchées en même temps
}
```

4.1 Type d'effet

La clé `type` détermine ce que fait l'Action. Voici tous les types disponibles :

Clé	Type de valeur	Valeur par défaut	Description
damage	—	—	Inflige des dégâts directs aux cibles. La quantité de dégâts est définie par force.
slowness	—	—	Applique l'effet de Lenteur. force définit le niveau (1 = Lenteur I, 2 = Lenteur II...). timeForce définit la durée en ticks.
speed	—	—	Applique l'effet de Vitesse. Même fonctionnement que slowness.
blindness	—	—	Applique l'effet de Cécité. Même fonctionnement que slowness.
regen	—	—	Applique l'effet de Régénération. Même fonctionnement que slowness.

Clé	Type de valeur	Valeur par défaut	Description
fireResistance	—	—	Applique l'effet de Résistance au feu. Même fonctionnement que slowness.
fire	—	—	Met les cibles en feu. La durée est définie par timeForce (en ticks). force n'est pas utilisé.
invicibility	—	—	Rend les cibles invincibles temporairement. La durée est définie par timeForce (en ticks). Techniquement, définit les ticks d'invincibilité après un coup reçu.
tp	—	—	Téléporte le lanceur vers une cible choisie aléatoirement parmi les joueurs ciblés par l'Action.
jauge	—	—	Incrémente la jauge nommée dans la clé jauge de la valeur jaugeAdd, pour chaque joueur ciblé.
heal	force = PV soignés	—	Soigne instantanément les cibles de force points de vie.
conditionChanged	—	—	Inverse la valeur d'une condition globale de la partie. Le nom de la condition à modifier se renseigne dans la clé conditionChanged de l'Action (pas dans type). Utilise force et timeForce : ignorés. N'affecte pas les joueurs ciblés.
changeNumConds	—	—	Modifie une ou plusieurs conditions numériques. Les modifications sont dans la clé numConditionsChanged de l'Action : tableau de { "numConditionChanged": "nomCond", "numConditionChange": N } où N est la valeur ajoutée (négative pour décrémenter). N'affecte pas les joueurs ciblés.
spectator	timeForce = durée	—	Passe les joueurs ciblés en mode spectateur. Après timeForce secondes, ils sont automatiquement remis en mode survie. Suit les règles de ciblage normales (distance, campTargeted, etc.) — peut affecter plusieurs joueurs. force est ignoré.
grrr	—	—	Joue le son de grognement de loup (WOLF_GROWL) pour chaque joueur ciblé, depuis la position du lanceur. force et timeForce ignorés.
hurlement	—	—	Joue le son de hurlement de loup (WOLF_HOWL) pour chaque joueur ciblé, depuis la position du lanceur. force et timeForce ignorés.
expulsion	force = intensité	—	Propulse les joueurs ciblés loin du lanceur (knockback). force détermine l'intensité de la propulsion.
attraction	force = intensité	—	Attire les joueurs ciblés vers la position du lanceur. force détermine l'intensité de l'attraction.
maxHealth	force = PV ajoutés	—	Augmente le maximum de points de vie des cibles de force. timeForce ignoré.
fly	force = niveau	timeForce = durée	Donne la capacité de vol aux cibles pendant timeForce secondes. À l'expiration, le vol est retiré et

Clé	Type de valeur	Valeur par défaut	Description
			un effet de saut (JUMP) est appliqué au niveau force pour la même durée (amortissage de chute).

Les effets de potion (slowness, speed, blindness, regen, fireResistance) ne remplacent pas un effet de même type déjà actif si ce dernier est de niveau supérieur. Comportement standard de Minecraft/Bukkit.

4.2 Intensité et durée

Clé	Type de valeur	Valeur par défaut	Description
force	Number	1.0	Intensité de l'effet. Pour damage : nombre de points de dégâts. Pour les potions (slowness, speed...) : niveau de l'effet (1 = niveau I, 2 = niveau II...). Ignoré par fire, invicibility et tp.
timeForce	Number	1.0	Durée en ticks pour les effets de potion, fire et invicibility. 20 ticks = 1 seconde. Exemple : 100 ticks = 5 secondes. Ignoré par damage, tp et jauge.

4.3 Probabilité de déclenchement

Clé	Type de valeur	Valeur par défaut	Description
proba	Integer	100	Probabilité en % que l'Action se déclenche. 100 = toujours. 50 = une fois sur deux. 0 = jamais. Cette vérification est faite en tout premier : si le dé rate, aucun effet n'est appliqué, même les actions imbriquées.
conditions	Object	aucune	Filtre conditionnel. Si renseigné, chaque paire clé/valeur est vérifiée contre les conditions définies dans le gamemode. Si l'une ne correspond pas, l'action est annulée entièrement. Exemple : { "porteOuverte": true } empêche l'action si la condition "porteOuverte" est false dans la partie.
withoutMessage	Boolean	false	Si true, aucun message n'est envoyé au lanceur après l'action (pas de rapport de déclenchement). Utile pour des actions silencieuses ou déclenchées en arrière-plan. Attention : la valeur par défaut est true — les messages sont désactivés par défaut. Mettre false pour les activer.
numConditions	Array	aucune	Filtre numérique. Tableau d'objets { "name": "nomCond", "value": N, "comparator":

Clé	Type de valeur	Valeur par défaut	Description
			"more" "less" "equals" }. Si une condition ne satisfait pas le comparateur, l'Action est annulée. Exemple : [{"name":"kills","value":3,"comparator":"more"}] → ne se déclenche que si kills > 3.

4.4 Qui est ciblé ?

Les propriétés suivantes déterminent quels joueurs sont affectés par l'Action. Elles sont évaluées dans l'ordre indiqué ci-dessous.

Clé	Type de valeur	Valeur par défaut	Description
onlySelf	Boolean	false	Si true : seul le lanceur de l'Action est ciblé, quelle que soit la configuration des autres propriétés. Priorité maximale.
commandTarget	Boolean	false	Si true : la seule cible est le joueur dont le nom est passé en premier argument de la commande (/lg <cmd> <joueur>). Ne fonctionne que dans une commande.
multipleCommandTarget	Boolean	false	Si true : les cibles sont tous les joueurs dont les noms sont passés comme arguments (/lg <cmd> <j1> <j2>...). Ne fonctionne que dans une commande.
distance	Integer	1000	Rayon en blocs autour du lanceur. Seuls les joueurs dans ce rayon sont éligibles. Ignoré si onlySelf ou commandTarget est actif.
self	Boolean	false	Si true : le lanceur lui-même est inclus parmi les cibles potentielles du rayon. Par défaut, il est exclu.
campTargeted	String	"null"	Si renseigné (valeur différente de "null") : parmi les cibles éligibles, seuls les joueurs appartenant à ce camp sont retenus.
roleTargeted	String	"null"	Si renseigné : parmi les cibles éligibles, seuls les joueurs ayant exactement ce nom de rôle sont retenus.

Priorité de ciblage : onlySelf > commandTarget > multipleCommandTarget > rayon (distance + self). Les filtres campTargeted et roleTargeted s'appliquent ensuite sur la liste résultante.

⚠ commandTarget et multipleCommandTarget n'ont d'effet que dans les actions déclenchées par une commande (tableau commands du rôle). Dans attackAction ou deathAction, ils sont ignorés.

4.5 Incrémenter une jauge

Clé	Type de valeur	Valeur par défaut	Description
jauge	String	"null"	Nom de la jauge à incrémenter (doit correspondre au nom d'une jauge déclarée dans ce rôle). Si "null", cette propriété est ignorée.
jaugeAdd	Integer	0	Valeur ajoutée à la jauge pour chaque joueur ciblé. Utilisé uniquement avec le type "jauge".
numConditionsChanged	Array	aucune	Utilisé avec le type changeNumConds. Tableau de { "numConditionChanged": "nomCond", "numConditionChange": N } où N est la valeur ajoutée à la condition numérique (négatif pour décrémenter).

4.6 Actions imbriquées

Clé	Type de valeur	Valeur par défaut	Description
actions	Array (Action[])	aucune	Liste d'Actions supplémentaires déclenchées simultanément. Permet de combiner plusieurs effets en un seul bloc. Chaque élément est un objet Action complet.
delayedActions	Array	aucune	Liste d'Actions à déclencher avec un délai. Chaque élément est un objet avec deux clés : "action" (un objet Action complet) et "time" (délai en secondes avant le déclenchement). Contrairement à actions, les delayedActions ne sont pas instantanées.
repeat	Integer	-1	Délai en secondes entre chaque répétition de l'Action. Doit être combiné avec repeatMax. Si -1 (ou absent), l'Action ne se répète pas.
repeatMax	Integer	-1	Nombre de répétitions supplémentaires après le premier déclenchement. Décrémenté à chaque fois jusqu'à 0. Exemple : repeat: 20, repeatMax: 3 → l'action se redéclenche 3 fois toutes les 20 secondes.
repeat	Integer	-1 (inactif)	Délai en secondes avant de rejouer l'action. Doit être utilisé avec repeatMax. Si les deux sont positifs, l'action se re-déclenche automatiquement après repeat secondes, repeatMax fois au total.
repeatMax	Integer	-1 (inactif)	Nombre de répétitions restantes. Décrémenté à chaque re-déclenchement. Quand il atteint 0, la répétition s'arrête. Doit être utilisé avec repeat.

```
// Exemple : infliger des dégâts ET ralentir en même temps
{
  "type": "damage",
  "force": 3,
  "distance": 10,
  "actions": [
```

```

    {
      "type": "slowness",
      "force": 1,
      "timeForce": 60,
      "distance": 10
    }
  ]
}

```

4.7 Exemples pratiques

Dégâts en zone à la mort (deathAction)

```

// Inflige 6 dégâts à tous les joueurs dans un rayon de 20 blocs à la mort
"deathAction": {
  "type": "damage",
  "force": 6,
  "distance": 20,
  "proba": 100
}

```

Ralentissement sur soi-même à chaque attaque (attackAction)

```

// Le joueur se ralentit lui-même pendant 3 s, seulement 50 % du temps
"attackAction": {
  "type": "slowness",
  "force": 1,
  "timeForce": 60,
  "onlySelf": true,
  "proba": 50
}

```

Commande ciblant un joueur par son nom

```

// /lg drain <joueur> : inflige 4 dégâts au joueur nommé et se regen de 4
"commands": [{
  "command": "drain",
  "action": {
    "type": "damage",
    "force": 4,
    "commandTarget": true,
    "proba": 100,
    "actions": [
      {
        "type": "heal",
        "force": 4,
        "onlySelf": true
      }
    ]
  }
}]

```

Commande ciblant plusieurs joueurs d'un camp adverse

```
// /lg maudit <j1> <j2> : aveugle les joueurs listés, s'ils sont du camp
"Vertueux"
{
  "command": "maudit",
  "action": {
    "type": "blindness",
    "force": 1,
    "timeForce": 80,
    "multipleCommandTarget": true,
    "campTargeted": "Vertueux"
  }
}
```

Téléportation vers un ennemi proche

```
// Téléporte le joueur vers un ennemi du camp "Maléfiques" dans 30 blocs
{
  "type": "tp",
  "distance": 30,
  "campTargeted": "Maléfiques"
}
```

Incrémenter une jauge à chaque attaque

```
// attackAction : ajoute 1 point à la jauge "ames" du lanceur à chaque attaque
{
  "type": "jauge",
  "jauge": "ames",
  "jaugeAdd": 1,
  "onlySelf": true
}
```

5. L'objet Jauge

Une Jauge est un compteur associé à un rôle. Chaque joueur qui possède ce rôle a son propre compteur, indépendant de celui des autres joueurs. Lorsque le compteur atteint un seuil (maxValue), une Action se déclenche automatiquement.

Cas d'usage typique : un personnage qui accumule de l'énergie à chaque attaque, et se régénère automatiquement toutes les 5 attaques.

5.1 Déclaration dans le rôle

```
"jauges": [
  {
    "name": "ferveur", // Identifiant de la jauge
    "defaultValue": -1, // Valeur de départ
    "maxValue": 5, // Seuil de déclenchement
    "resetOnFinish": true, // Se réinitialise après chaque déclenchement
    "iteration": 3, // Se déclenche 3 fois au maximum
    "action": { // Action déclenchée quand le seuil est
      atteint
        "type": "regen",
        "force": 1,
        "timeForce": 100,
        "onlySelf": true
      }
    }
  ]
```

5.2 Toutes les propriétés

Clé	Type de valeur	Valeur par défaut	Description
name	String	"null"	Identifiant unique de la jauge dans ce rôle. C'est cette valeur que tu dois renseigner dans la clé jauge d'une Action pour incrémenter cette jauge.
max	Integer	0	Valeur initiale du compteur au démarrage de la partie.
maxValue	Integer	100	Seuil de déclenchement. Lorsque le compteur atteint ou dépasse cette valeur, l'Action associée se déclenche.
resetOnFinish	Boolean	false	true = après chaque déclenchement, le compteur est réinitialisé (en conservant le surplus éventuel). false = le compteur se bloque définitivement après le premier déclenchement.
iteration	Integer	1	Nombre maximum de fois que la jauge peut se déclencher. Une fois ce nombre atteint, la jauge ne se déclenche plus jamais, même si le compteur continue d'être incrémenté.

Clé	Type de valeur	Valeur par défaut	Description
action	Object (Action)	aucune	L'Action déclenchée automatiquement lorsque le compteur atteint maxValue. Même format qu'une Action classique.
displayed	Boolean	false	Si true, la valeur actuelle de la jauge est affichée en temps réel au joueur dans la barre d'action (sous la barre de vie) à chaque incrémentation, sous la forme : NomJauge: valeur/maxValue.

5.3 Comment fonctionne le déclenchement ?

- À chaque incrémentation, le système vérifie si compteur + ajout $\geq$ seuil (maxValue).
- Si oui et que la jauge n'est pas bloquée : l'Action se déclenche, et le compteur d'itérations diminue de 1.
- Si le compteur d'itérations tombe à 0 : la jauge est bloquée définitivement (plus aucun déclenchement).
- Si resetOnFinish est true : le compteur repart à (compteur + ajout - seuil), en conservant le surplus.
- Si resetOnFinish est false : la jauge se bloque immédiatement après le premier déclenchement, peu importe iteration.
- Si compteur + ajout $<$ seuil : le compteur est simplement mis à jour, rien ne se passe.

 Pour une jauge qui se déclenche une fois puis s'arrête définitivement : `resetOnFinish: false, iteration: 1`. Pour une jauge qui se déclenche à l'infini : `resetOnFinish: true, iteration: 99999`.

 Si `resetOnFinish` est false, la jauge se bloque dès le premier déclenchement, quelle que soit la valeur de `iteration`.

5.4 Incrémenter une jauge depuis une Action

Pour faire évoluer une jauge, utilise une Action avec le type "jauge", la clé jauge (nom de la jauge) et `jaugeAdd` (valeur à ajouter). Cette Action peut être placée dans `attackAction`, dans une commande, etc.

```
// À chaque attaque, +1 dans la jauge "ferveur" du lanceur
{
  "type":      "jauge",
  "jauge":     "ferveur",
  "jaugeAdd":  1,
  "onlySelf": true
}
```

6. Exemple complet — Le Nécromancien

Ce rôle illustre toutes les fonctionnalités du plugin : résurrection conditionnelle, jauge cyclique alimentée par les attaques, action de zone à la mort, commandes avec ciblage précis et pouvoir d'espionnage.

```
{
  "name":          "Nécromancien",
  "camp":          "Maléfiques",
  "roleListName":  "classes",

  "strenght":      0.8,      // +80 % de dégâts infligés
  "resistance":    0.3,      // -30 % de dégâts reçus (très résistant)
  "boostHealth":   2,        // 2 PV bonus à l'attribution

  // Ressuscite 1 fois, uniquement si tué par un joueur du camp "Vertueux"
  "reliveTry":      1,
  "lostStrenght":   0.2,
  "lostResis":      0.1,
  "lostHealth":     4.0,
  "conditionKilledBy":  "",
  "conditionKilledByCamp": "Vertueux",

  // À chaque attaque : +1 dans la jauge "ames"
  "attackAction": {
    "type": "jauge", "jauge": "ames", "jaugeAdd": 1, "onlySelf": true
  },

  // À la mort : aveugle tous les joueurs dans 15 blocs
  "deathAction": {
    "type": "blindness", "force": 1, "timeForce": 100, "distance": 15
  },

  "commands": [
    {
      // /lg drain <joueur> : inflige 3 dégâts à ce joueur
      "command": "drain",
      "action": { "type": "damage", "force": 3, "commandTarget": true }
    },
    {
      // /lg plague <j1> <j2>... : ralentit ces joueurs si du camp "Vertueux"
      "command": "plague",
      "action": {
        "type": "slowness", "force": 2, "timeForce": 80,
        "multipleCommandTarget": true, "campTargeted": "Vertueux"
      }
    }
  ],

  // Jauge "ames" : toutes les 5 attaques → régénération pendant 4 s (jusqu'à 99
  // fois)
  "jauges": [
    {
      "name": "ames", "max": 0, "maxValue": 5,
      "resetOnFinish": true, "iteration": 99,
      "action": { "type": "regen", "force": 1, "timeForce": 80, "onlySelf": true }
    }
  ]
}
```

```
    }  
  ],  
  
  // Pouvoir : espionner le camp d'un joueur via /lg voir <joueur> (3 fois)  
  "infoPowers": [  
    {  
      "type": "seeCaract",  
      "object": {  
        "distance": 40, "probaTrue": 80, "use": 3,  
        "command": "voir", "knowWho": true, "caract": "camp"  
      }  
    }  
  ]  
}
```

7. Récapitulatif et rappels

7.1 Emplacements des fichiers

<code>Gamemodes/<nom>.json</code>	Fichier gamemode. Configure la partie (minage, groupes de rôles).
<code>plugins/<Plugin>/roles/<nom>.json</code>	Fichier rôle. Un fichier = un rôle. Tu peux en créer autant que tu veux.

7.2 Les types d'Action en un coup d'œil

Clé	Type de valeur	Valeur par défaut	Description
<code>damage</code>	force = dégâts	timeForce = ignoré	Dégâts directs.
<code>slowness</code>	force = niveau	timeForce = durée	Lenteur. Ne remplace pas un effet supérieur déjà actif.
<code>speed</code>	force = niveau	timeForce = durée	Vitesse. Ne remplace pas un effet supérieur déjà actif.
<code>blindness</code>	force = niveau	timeForce = durée	Cécité. Ne remplace pas un effet supérieur déjà actif.
<code>regen</code>	force = niveau	timeForce = durée	Régénération. Ne remplace pas un effet supérieur déjà actif.
<code>fireResistance</code>	force = niveau	timeForce = durée	Résistance au feu. Ne remplace pas un effet supérieur déjà actif.
<code>fire</code>	force = ignoré	timeForce = durée	Met en feu.
<code>invicibility</code>	force = ignoré	timeForce = durée	Invincibilité temporaire.
<code>tp</code>	force = ignoré	timeForce = ignoré	Téléporte le lanceur vers une cible aléatoire dans la liste.
<code>jauge</code>	force = ignoré	timeForce = ignoré	Incrémente la jauge désignée de <code>jaugeAdd</code> .
<code>heal</code>	force = PV soignés	—	Soigne instantanément les cibles de force points de vie.
<code>conditionChanged</code>	—	—	Inverse la valeur d'une condition globale. Le nom de la condition à modifier se renseigne dans la clé <code>conditionChanged</code> de l'Action.
<code>changeNumConds</code>	—	—	Incrémente/décrémente une ou plusieurs conditions numériques via <code>numConditionsChanged</code> .

Clé	Type de valeur	Valeur par défaut	Description
spectator	force = ignoré	timeForce = durée (s)	Spectateur temporaire pour les cibles. Remet en survie après timeForce secondes.
grrr	force = ignoré	timeForce = ignoré	Grognement de loup pour les cibles.
hurlement	force = ignoré	timeForce = ignoré	Hurlement de loup pour les cibles.
expulsion	force = intensité	—	Knockback : propulse les cibles loin du lanceur.
attraction	force = intensité	—	Attire les cibles vers le lanceur.
maxHealth	force = PV ajoutés	—	Augmente les PV max des cibles de force.
fly	force = niveau saut	timeForce = durée	Vol pendant timeForce secondes, puis effet JUMP niveau force (amortissage).

7.3 Rappels importants

- strenght et resistance : 1.0 = normal, 1.5 = +50 % de dégâts, 0.5 = -50 % de dégâts.
- proba est évalué en tout premier. Si le dé rate, rien ne se passe, même les actions imbriquées.
- onlySelf a la priorité absolue sur tous les autres filtres de ciblage.
- commandTarget et multipleCommandTarget n'ont d'effet que dans les actions d'une commande, pas dans attackAction ni deathAction.
- Une jauge avec resetOnFinish: false se bloque définitivement après son premier déclenchement.
- Pour appartenir à une liste de rôles, un fichier de rôle doit avoir le bon roleName, correspondant au listName déclaré dans le gamemode.
- Avec N listes dans le gamemode, chaque joueur reçoit exactement N rôles (un par liste). Leurs effets se cumulent.